Network Optimal Control and Differential Games

Models, necessary and sufficient conditions, hybrid decisions, and Python examples

Lu-Xing Yang

This note develops optimal control and differential games for propagation on networks. It begins with empirical degree distributions and incoming-influence matrices, derives continuous and impulsive Pontryagin conditions, connects them to HJB and feedback games, and explains how to run and adapt the accompanying code. The central examples remain small enough to follow by hand while retaining several degree classes or network nodes. Prerequisites are multivariable calculus, linear algebra, elementary ordinary differential equations, and basic Python.

How to use this note

Read the model and the mathematical derivation before changing the code. Run the first example, inspect its state and decision curves, and then use the worked extensions. The equations, assumptions and interpretation are included here; the repository supplies the executable implementations.

Contents

1 The modeling question and the learning route **4**
1.1 Research context: from cyber dynamics to interventions . . . 4
1.2 Notation and units . . . 4
1.3 Three model resolutions . . . 5

2 From external network data to model inputs **5**
2.1 Read the graph and preserve node identities . . . 5
2.2 Compute the empirical degree distribution first . . . 6
2.3 Derive the uncorrelated degree closure . . . 6
2.4 Retain degree correlations when they matter . . . 6

3 Controlled propagation models **7**
3.1 Degree-level and node-level SIS dynamics . . . 7
3.2 Patching and cleaning are different transitions . . . 7
3.3 Objectives, population weights and shared resources . . . 8
3.4 An optional connection to a propagation threshold . . . 8

4 Pontryagin's principle: derivation and interpretation **8**
4.1 The continuous-time problem . . . 8
4.2 Bounds and projected optimality . . . 9
4.3 When necessary conditions become sufficient . . . 10

5 Derive the network-control equations **10**
5.1 Degree-class costates and controls . . . 10
5.2 Node-level costates and controls . . . 11
5.3 A shared budget changes the update . . . 11
5.4 A different intervention requires a different derivative . . . 11

6 Numerical solution and the first executable example **11**
6.1 Forward–backward sweep . . . 11
6.2 Install and run . . . 12
6.3 Follow one code path . . . 12
6.4 Interpret the outputs . . . 13

7 Differential games on the same network **13**
7.1 Specify the players and the information structure . . . 13
7.2 A degree-class attacker–defender game . . . 13
7.3 What a numerical game iteration establishes . . . 14

8 Dynamic programming, HJB and feedback verification **14**
8.1 The value function and Bellman's argument . . . 14
8.2 Zero-sum and general-sum feedback games . . . 15

9 Impulses, switches and hybrid control **15**
9.1 Describe timing, action values and state effects separately . . . 15
9.2 Costate jump: derive it from the reset . . . 15
9.3 A freely chosen event time . . . 16
9.4 Impulse dynamic programming and the QVI . . . 16
9.5 What is optimized in a hybrid computation? . . . 16

10 Extend the model while keeping one mathematical meaning **17**
10.1 Change five connected components together . . . 17
10.2 Research code as a next step . . . 17
10.3 A practical reading order for the tutorial series . . . 17

11 Research examples: connect each method to its modeling purpose **18**
11.1 APT evaluation, patching, repair and recovery . . . 18
11.2 Conflicting opinions and rumor refutation . . . 18
11.3 Propaganda war: degree classes, two players and hybrid strategies . . . 18
11.4 Awareness and coupled malware–opinion dynamics . . . 19
11.5 Feedback control: update a model as new information arrives . . . 19

A Short exercises with worked answers **19**

B Where to run extensions and numerical checks **20**

1 The modeling question and the learning route

A network intervention changes a process that is already evolving: infected devices recover, vulnerable devices are patched, competing messages spread, or a campaign produces an immediate change in awareness. A useful control model connects four objects: the state, the intervention mechanism, the information available to the decision maker, and the objective. This note follows that connection from a graph to differential equations, from equations to optimality conditions, and from conditions to executable examples.

The first example allocates recovery effort among degree classes. The same construction then extends to individual nodes, two strategic players, and interventions that cause jumps. Classical optimal-control results are developed using the minimization convention $H = L + \lambda^\top f$ [1, 2, 3]. Payoff-maximizing games are identified explicitly when introduced.

1.1 Research context: from cyber dynamics to interventions

The examples in this tutorial grow out of a line of cyber/network modeling and control research by Lu-Xing Yang and collaborators. Security evaluation under advanced persistent threats provides a starting point: derive the evolving attack–defense state, then connect its long-run behavior to a security measure [8]. Introducing strategic intervention leads to patch-injection and repair games [9, 10]; introducing recoverable data changes the available defense mechanisms and leads to backup-and-recovery decisions [11]. These are distinct modeling questions even when their optimality systems have the same state–costate form.

The social-network branch develops objectives for conflicting opinions and rumor refutation [12, 13]. Hybrid propaganda competition then combines sustained effort with discrete campaigns [17], while awareness-dependent propaganda suppression and coupled malware–opinion dynamics connect behavior with impulse interventions [18, 19]. Impulsive artificial defense and defense outsourcing provide related APT settings in which event timing and the allocation of decision authority matter [14, 15]. Finally, feedback-loop malware control links parameter learning with model predictive control (MPC) [16].

The compact SIS/SIR equations below are teaching models that isolate those mathematical ingredients. The later research examples explain how to move from them to the cited publications and their code. A tutorial illustration is identified separately from a paper's original model or algorithm; changing the physical state, information pattern or cost creates a different problem.

1.2 Notation and units

Symbol	Meaning
n, i, j	Number of nodes and node indices.
$k, \mathcal{K}$	An observed degree and the set of observed degree classes.
$P(k), \bar{k}$	Fraction of nodes in degree class k and average degree.
A_{ij}	Incoming influence: node j affects node i.
x, u	State vector and continuous-valued control. Their dimensions depend on the model.
f, L, Φ	State dynamics, running cost, and terminal cost.
λ, H	Costate vector and Hamiltonian.
t_r, τ_ℓ	A sampled decision time and an impulse time; these need not coincide.
G, K	State-reset map and instantaneous intervention cost.

A state such as $x_i \in [0, 1]$ is an infection probability or mean-field fraction, not a binary observation of a particular host. The expected infected population is $\sum_i x_i$. A degree-class state x_k is the infected fraction *within* class k, and the population fraction is $\sum_k P(k)x_k$. If time is measured in hours, transmission, natural recovery and recovery-control intensities have units of inverse hours. An impulse fraction is dimensionless; its cost is charged at an event rather than multiplied by the duration of a numerical step.

1.3 Three model resolutions

An aggregate compartment model describes a population by a few states, such as S, I, R. A degree-level model keeps a separate state for every observed degree. A node-level model keeps a separate state for every node and uses the adjacency matrix directly. These are distinct approximations. An aggregate SIR system does not become degree-level merely by storing its three compartments in an array.

The degree model reduces dimension when the number of observed degrees is much smaller than n. The node model retains location, direction and weighted contacts. A degree closure can lose correlations between a node's state and its neighbors; a node mean-field model still approximates stochastic dependencies between individual infection events. Model resolution should therefore follow the scientific question rather than the available array size.

2 From external network data to model inputs

2.1 Read the graph and preserve node identities

The introductory command accepts a whitespace-separated edge list, a CSV edge list, a numeric adjacency CSV, or a supported NetworkX graph file. For example:

Listing 1: CSV edge-list structure

```
source,target,weight
001,002,1.0
002,003,0.5
003,001,1.0
```

Identifiers such as `001` are labels, not numbers to be rounded or renumbered implicitly. The repository reader stores them as text. Repeated edge records are summed, including reversed records of an undirected edge. Self-loops are removed, zero-weight edges create no contact, and negative or nonfinite weights are rejected. An edge list cannot describe a node that appears nowhere in the data; use GraphML, a numeric adjacency matrix, or an explicit node list when isolates matter.

A graph-to-matrix conversion must use a fixed node order. NetworkX stores a directed edge from source i to target j in position (i, j); the incoming-influence convention used here requires its transpose [6]:

Listing 2: Construct an incoming-influence matrix

```
import networkx as nx
import numpy as np

nodes = list(graph.nodes())
A = nx.to_scipy_sparse_array(
    graph, nodelist=nodes, weight="weight", format="csr"
).T.tocsr()
x = np.full(len(nodes), 0.05)
pressure = A @ x
# Save nodes alongside any node-indexed trajectories.
```

For an undirected graph the transpose makes no numerical difference. For a directed graph it determines which nodes receive infection pressure. For example, an edge $j \to i$ must contribute $A_{ij}x_j$ to node i.

2.2 Compute the empirical degree distribution first

For the full, simple, undirected input graph, compute

$$N_k = \#\{i : \deg(i) = k\}, \qquad P(k) = \frac{N_k}{n}, \qquad \bar{k} = \sum_{k \in \mathcal{K}} kP(k). \tag{1}$$

Only observed classes are stored; $k = 0$ is included when isolates exist. A compact calculation is

Listing 3: Degree classes are columns of the state array

```
degrees = np.fromiter((d for _, d in graph.degree()), dtype=int)
k, N_k = np.unique(degrees, return_counts=True)
p = N_k / N_k.sum()
kbar = float(k @ p)
# X.shape == (number_of_times, len(k))
# X[:, j] is the infected fraction for degree k[j].
```

Here degree counts distinct neighbors. Weighted strength, $\sum_j A_{ij}$, is a different quantity. The repository function `degree_distribution_from_graph` returns these arrays together in a `DegreeData` object. Degree statistics are computed before any node-level subgraph is selected, and computing them does not require a dense $n \times n$ matrix.

2.3 Derive the uncorrelated degree closure

Choose a random edge endpoint in an undirected network. Class k contributes kN_k endpoints, so its endpoint probability is $kP(k)/\bar{k}$. Under the uncorrelated degree approximation, the probability that a neighbor is infected is

$$\Theta(t) = \sum_{k \in \mathcal{K}} \frac{kP(k)}{\bar{k}} x_k(t). \tag{2}$$

Thus a susceptible degree-k node has expected infected-neighbor count $k\Theta$. For an edgeless graph, set $\Theta = 0$; the state then evolves through recovery and intervention only. No division by zero is needed.

The closure uses unweighted contacts. Reading and retaining edge weights does not make this particular degree equation weight-aware. Likewise, replacing k by in-degree is not, by itself, a derivation for a directed network. Direction, weight, degree correlation, and neighbor-state correlation require a suitable extension or a node-level model.

2.4 Retain degree correlations when they matter

Let $M_{k\ell}$ be the conditional probability that a neighbor of a degree-k node has degree ℓ. The correlated pressure is

$$\Theta_k(t) = \sum_{\ell \in \mathcal{K}} M_{k\ell} x_\ell(t). \tag{3}$$

For an empirical undirected graph, let $E_{k\ell}$ count oriented edge endpoints from class k to class ℓ: every undirected edge contributes both orientations. Then

$$M_{k\ell} = \frac{E_{k\ell}}{kN_k} \quad (k > 0), \qquad M_{0\ell} = 0. \tag{4}$$

Non-isolate rows sum to one, and the endpoint counts satisfy

$$kP(k)M_{k\ell} = \ell P(\ell) M_{\ell k}. \tag{5}$$

This balance relation is useful when inventing a mixing matrix: an arbitrary row-stochastic matrix need not correspond to an undirected graph with the stated degree distribution. The uncorrelated approximation

sets every non-isolate row to $\ell P(\ell)/\bar{k}$. Empirical mixing preserves degree correlations, but still replaces individual neighbor states by their class means.

A matrix transformation is a modeling choice

Row normalization replaces contact counts by normalized influence. Multiplying the whole matrix by a constant changes the transmission scale unless the rate is adjusted accordingly. Selecting a high-degree induced subgraph removes contacts and defines a different dynamical system. State these choices when interpreting a result; none is merely a formatting operation.

3 Controlled propagation models

3.1 Degree-level and node-level SIS dynamics

The first model uses recovery control:

$$\dot{x}_k = \beta k(1 - x_k)\Theta - (\delta + u_k)x_k, \qquad 0 \le u_k \le \bar{u}_k. \tag{6}$$

Here β is the per-contact transmission rate, δ is natural recovery, and u_k is extra recovery/removal effort. A corresponding node-level model is

$$\dot{x}_i = \beta_i(1 - x_i)\sum_j A_{ij}x_j - (\delta_i + u_i)x_i. \tag{7}$$

All rates and matrix entries are nonnegative. At $x_i = 0$, the derivative is nonnegative; at $x_i = 1$, it equals $-(\delta_i + u_i) \le 0$. Therefore the vector field points inward on the boundary of $[0, 1]^n$. The same argument applies to degree classes. This property belongs to the differential equations. Large numerical clipping corrections would indicate a numerical problem rather than establish the property.

Heterogeneity can be represented by class- or node-specific susceptibility, infectivity, recovery, costs and bounds. A useful degree form is

$$\dot{x}_k = s_k k(1 - x_k)\sum_\ell M_{k\ell}b_\ell x_\ell - (\delta_k + u_k)x_k. \tag{8}$$

The acquisition coefficient s_k includes the transmission-rate scale; b_ℓ describes relative infectivity. In `DegreeSISParams`, these correspond to `susceptibility` and `infectivity`. Passing $s_k = \beta$ and $b_\ell = 1$ recovers (6) under uncorrelated mixing.

3.2 Patching and cleaning are different transitions

To distinguish protection from recovery, use a controlled SIR/SIPS-type model:

$$\begin{aligned} \dot{S} &= -\beta SI - u_p S + \omega R, \\ \dot{I} &= \beta SI - (\gamma + u_c)I, \\ \dot{R} &= u_p S + (\gamma + u_c)I - \omega R. \end{aligned} \tag{9}$$

Patching u_p moves $S \to R$, whereas cleaning u_c moves $I \to R$. The sum of the equations is zero. In the SIS example, recovered devices immediately rejoin susceptibility, so its recovery control should not be described as SIR patching. This distinction also determines the Hamiltonian derivative: patching has a marginal state effect proportional to S, while cleaning has one proportional to I.

3.3 Objectives, population weights and shared resources

For degree-level control, use

$$J(u) = \int_0^T \sum_k P(k) \left[q_k x_k + \frac{r_k}{2} u_k^2 \right] \mathrm{d}t + \Phi(x(T)), \qquad r_k > 0. \tag{10}$$

This is a per-capita weighted objective. Multiplying the entire objective by n leaves its minimizer unchanged. Multiplying only the state term by n changes the tradeoff. A node-level objective may instead sum host-specific risks and intervention costs, or divide the entire sum by n; make that choice consistently in the solver and every baseline.

Independent bounds model separate capacity limits. A shared budget, such as

$$\sum_k c_k u_k(t) \le B(t), \qquad c_k > 0, \tag{11}$$

couples the decisions. The meaning of c_k must specify whether effort is charged per class, per node, or by class population. Independent clipping generally does not enforce this budget.

3.4 An optional connection to a propagation threshold

For a fixed control u and node dynamics $\dot{x} = (I - \operatorname{diag} x)Bx - Dx$, define $B_{ij} = \beta_i A_{ij}$ and $D = \operatorname{diag}(\delta_i + u_i)$, with all diagonal entries of D positive. Linearization at zero is $\dot{z} = (B - D)z$.

Proposition 3.1 (A sufficient extinction condition)**.** *If $\rho(D^{-1}B) < 1$, the disease-free state is globally exponentially stable on $[0, 1]^n$.*

Proof. The matrix $B - D$ is Metzler and Hurwitz under the stated spectral condition. Since $\dot{x} \le (B - D)x$ componentwise, comparison with the positive linear system gives $0 \le x(t) \le e^{(B-D)t}x(0)$, which decays exponentially. □

For the homogeneous, uncontrolled degree closure, the rank-one next-generation matrix gives

$$\mathcal{R}_0 = \frac{\beta \sum_k k^2 P(k)}{\delta \bar{k}}. \tag{12}$$

This explains why the second degree moment can matter even at the same average degree. The strict threshold concerns long-run persistence under fixed coefficients. Finite-horizon control also depends on current infections, the remaining time and costs; a spectral ranking and an optimal finite-horizon allocation need not agree. The equality case is not classified by the strict comparison argument above.

4 Pontryagin's principle: derivation and interpretation

4.1 The continuous-time problem

Consider

$$\min_{u(\cdot)} J(u) = \Phi(x(T)) + \int_0^T L(x, u, t)\, \mathrm{d}t, \qquad \dot{x} = f(x, u, t), \quad x(0) = x_0, \quad u(t) \in U. \tag{13}$$

Assume a fixed horizon, a fixed initial state, a free terminal state, suitable smoothness in x, and admissible measurable controls for which solutions exist. The presentation treats the normal case and no additional active state-path constraints. Endpoint constraints, state constraints and abnormal extremals require their corresponding multipliers and transversality conditions [1, 2].

Define

$$H(x, u, \lambda, t) = L(x, u, t) + \lambda^\top f(x, u, t). \tag{14}$$

Introduce a multiplier for the differential equation:

$$\mathcal{A} = \Phi(x(T)) + \int_0^T \left[H(x, u, \lambda, t) - \lambda^\top \dot{x}\right] \, \mathrm{d}t. \tag{15}$$

For a state variation z with $z(0) = 0$, integration by parts gives

$$\begin{aligned} \delta\mathcal{A} = {} & [\Phi_x(x(T)) - \lambda(T)]^\top z(T) \\ & + \int_0^T (H_x + \dot{\lambda})^\top z \, \mathrm{d}t + \int_0^T H_u^\top \delta u \, \mathrm{d}t. \end{aligned} \tag{16}$$

Choosing the costate to cancel state variations yields the adjoint and terminal equations. Needle variations supply the pointwise Hamiltonian minimum condition; an ordinary differentiable variation supplies its first-order form on a convex control set.

Theorem 4.1 (Normal PMP conditions)**.** *An optimal pair in the stated setting has a costate satisfying, almost everywhere,*

$$\begin{aligned} \dot{x}^* &= f(x^*, u^*, t), \qquad x^*(0) = x_0, \\ \dot{\lambda}^* &= -H_x(x^*, u^*, \lambda^*, t), \qquad \lambda^*(T) = \Phi_x(x^*(T)), \\ u^*(t) &\in \arg\min_{v \in U} H(x^*(t), v, \lambda^*(t), t). \end{aligned} \tag{17}$$

These are necessary conditions. The state equation runs forward from 0*; the costate equation runs backward from* T*.*

A costate measures first-order sensitivity of future cost along an extremal. Identifying it with the gradient of a value function requires the differentiability and optimality assumptions of the value-function formulation; it is not an automatic interpretation of every numerically generated adjoint.

4.2 Bounds and projected optimality

For a differentiable Hamiltonian and a convex control set, its first-order necessary condition is

$$H_u(u^*)^\top (v - u^*) \geq 0 \qquad \text{for all } v \in U. \tag{18}$$

For a scalar interval $[0, \bar{u}]$, this reads

$$H_u = 0 \text{ in the interior}, \qquad H_u \geq 0 \text{ at } u = 0, \qquad H_u \leq 0 \text{ at } u = \bar{u}. \tag{19}$$

Thus penalizing H_u^2 at every time would penalize some correct boundary solutions. For any $\eta > 0$, the equivalent projected first-order residual is

$$r_u = u - \mathrm{Proj}_U(u - \eta H_u). \tag{20}$$

If H is convex in u, a zero residual is sufficient for its pointwise minimum. It still does not establish global optimality of the full nonlinear trajectory problem.

When $H(u) = ru^2/2 - bu +$ constant with $r > 0$, its minimizer is $u^* = \mathrm{clip}(b/r, 0, \bar{u})$. Taking $\eta = 1/r$ in (20) gives exactly $r_u = u - u^*$.

4.3 When necessary conditions become sufficient

Theorem 4.2 (A convexity sufficiency result)**.** *Suppose a candidate satisfies* (17)*. Suppose* Φ *is convex and, for almost every* t*,* $H(x, u, \lambda^*(t), t)$ *is jointly convex in* (x, u) *on a convex domain containing all compared admissible pairs. Then the candidate is globally optimal for* (13)*.*

Proof. For an admissible comparison pair, write $z = x - x^*$. Integration by parts using the costate equation gives

$$J(u) - J(u^*) = \Phi(x(T)) - \Phi(x^*(T)) - \lambda^*(T)^\top z(T) + \int_0^T \left[H(x, u, \lambda^*, t) - H(x^*, u^*, \lambda^*, t) - H_x^\top z\right] \mathrm{d}t.$$

Terminal convexity makes the first line nonnegative. Joint Hamiltonian convexity bounds the integrand below by $H_u^\top (u - u^*)$, which is nonnegative by (18). □

Nonlinear propagation includes products of states and controls. A positive quadratic effort cost makes the Hamiltonian convex in each control, but does not automatically make it jointly convex in states and controls. For these examples, the sweep usually produces a PMP candidate; a separate verification argument would be needed for a global guarantee.

Example 4.3 (A transparent sufficient case)**.** For $\dot{x} = u$ and $J = \frac{1}{2}sx(T)^2 + \frac{1}{2}\int_0^T (x^2 + ru^2)\,\mathrm{d}t$ with $s \geq 0, r > 0$, PMP gives $\dot{\lambda} = -x$, $u = -\lambda/r$, and $\lambda(T) = sx(T)$. Set $\lambda = P(t)x$. Then

$$-\dot{P} = 1 - P^2/r, \qquad P(T) = s, \qquad u = -P(t)x/r.$$

The Hamiltonian is jointly convex, so the resulting extremal is optimal. The quadratic value function $V(t, x) = P(t)x^2/2$ also verifies optimality through HJB.

5 Derive the network-control equations

5.1 Degree-class costates and controls

For (6), set $w_k = kP(k)/\bar{k}$ when $\bar{k} > 0$ and $w_k = 0$ otherwise. The Jacobian entries are

$$\frac{\partial f_k}{\partial x_\ell} = \beta k(1 - x_k)w_\ell - \mathbf{1}_{\{k=\ell\}}\left[\beta k\Theta + \delta + u_k\right]. \tag{21}$$

For $L = \sum_k P(k)[q_k x_k + r_k u_k^2/2]$, the costate equations become

$$\dot{\lambda}_\ell = -q_\ell P(\ell) + \lambda_\ell[\beta\ell\Theta + \delta + u_\ell] - w_\ell \sum_k \lambda_k \beta k(1 - x_k), \qquad \lambda(T) = \Phi_x(x(T)). \tag{22}$$

The sum over all classes is essential: changing one class changes the pressure experienced by every other class. Since $\partial f_k/\partial u_k = -x_k$,

$$H_{u_k} = r_k P(k)u_k - \lambda_k x_k, \qquad u_k^* = \mathrm{clip}\left(\frac{\lambda_k x_k}{r_k P(k)}, 0, \bar{u}_k\right). \tag{23}$$

All stored observed classes have $P(k) > 0$. Isolates have zero infection pressure but can still receive recovery effort if initially infected.

For the heterogeneous model, replace the rank-one transmission matrix by $B_{k\ell} = s_k k M_{k\ell} b_\ell$. The same matrix formula applies:

$$f = (1 - x) \odot (Bx) - (\delta + u) \odot x, \quad f_x = \mathrm{diag}(1 - x)B - \mathrm{diag}(Bx + \delta + u). \tag{24}$$

Here $\odot$ denotes componentwise multiplication. This form keeps the model, its Jacobian and its adjoint visibly aligned.

5.2 Node-level costates and controls

For (7), define $B = \mathrm{diag}(\beta_i)A$ and use the same f_x in (24). With $L = \sum_i [q_i x_i + r_i u_i^2/2]$,

$$\dot{\lambda} = -q - f_x^\top \lambda, \qquad u_i^* = \mathrm{clip}(\lambda_i x_i / r_i, 0, \bar{u}_i). \tag{25}$$

For large sparse networks, the adjoint only needs a Jacobian-vector product:

$$f_x^\top \lambda = B^\top [(1 - x) \odot \lambda] - (Bx + \delta + u) \odot \lambda. \tag{26}$$

No dense Jacobian is needed to evaluate this expression. Sparsity reduces computation without changing the model, unlike replacing the full network by a smaller induced subgraph.

5.3 A shared budget changes the update

At a fixed time, suppose the Hamiltonian control terms are $\sum_k [r_k P(k) u_k^2/2 - b_k u_k]$ and $\sum_k c_k u_k \le B$. A nonnegative multiplier μ gives

$$u_k(\mu) = \mathrm{clip}\left(\frac{b_k - \mu c_k}{r_k P(k)}, 0, \bar{u}_k\right). \tag{27}$$

If the unconstrained allocation is feasible, take $\mu = 0$. Otherwise choose μ so the budget is tight. The total expenditure is nonincreasing in μ, making a scalar bisection sufficient for this particular convex subproblem. More general coupled constraints require their own constrained Hamiltonian minimization.

5.4 A different intervention requires a different derivative

For transmission reduction,

$$f_k = \beta(1 - u_k)k(1 - x_k)\Theta - \delta x_k, \qquad 0 \le u_k \le 1, \tag{28}$$

we have $f_{k,u_k} = -\beta k(1 - x_k)\Theta$. Therefore

$$u_k^* = \mathrm{clip}\left(\frac{\lambda_k \beta k(1 - x_k)\Theta}{r_k P(k)}, 0, \bar{u}_k\right). \tag{29}$$

Its state Jacobian also changes. Keeping the old $\lambda_k x_k$ update after replacing the state equation would optimize the wrong intervention mechanism.

6 Numerical solution and the first executable example

6.1 Forward–backward sweep

Initialize an admissible control on a time grid. With that control held fixed, solve the state equation forward. Interpolate the state and control consistently, solve the costate equation backward, and form the pointwise Hamiltonian minimizer. Let U denote all control values on the grid and $\mathcal{T}(U)$ their candidate update. Compute

$$\varepsilon = \|\mathcal{T}(U) - U\|_\infty . \tag{30}$$

Stop when this residual is small. Otherwise relax,

$$U \leftarrow (1 - \alpha)U + \alpha \mathcal{T}(U), \qquad 0 < \alpha \le 1, \tag{31}$$

and repeat. A small relaxed step alone is inconclusive when α is small. The returned trajectory and objective must be replayed under the returned control, rather than under the previous iterate.

Forward–backward sweep is a fixed-point method; convergence requires conditions, and is not guaranteed for every nonlinear problem or horizon [4]. Grid refinement, tighter integration tolerances and alternative initial controls help distinguish discretization error from a difficult boundary-value problem. Direct transcription instead optimizes a finite-dimensional state/control parameterization subject to discretized dynamics. It is a useful comparison method, but its nonlinear optimizer may also return a local solution.

6.2 Install and run

Use a separate environment with Python 3.11 or 3.12. Run the following from the repository root:

```
python -m venv .venv
# macOS / Linux:
source .venv/bin/activate
# Windows PowerShell instead:
# .venv\Scripts\Activate.ps1
python -m pip install -e .
python -m cybercontrol
```

No external dataset, GPU or LaTeX installation is required for the default example. The command reads a seeded 40-node Barabasi–Albert teaching graph, computes its full degree distribution, and solves the scalar-parameter degree SIS example with $\beta = 0.12$, $\delta = 0.18$, $q = 3$, $r = 2.5$, $\bar{u} = 1.2$ and $T = 14$. These are illustrative rates and costs, not fitted empirical estimates. The extended solvers have their own heterogeneous profiles; those profiles should not be confused with the introductory CLI settings.

```
python -m cybercontrol --beta 0.15 --control-cost 4
python -m cybercontrol --edge-list data/edges.txt
python -m cybercontrol --edge-list data/edges.csv --delimiter , --has-header --
    source-col source --target-col target --weight-col weight
python -m cybercontrol --graph-file data/network.graphml
python -m cybercontrol --adjacency-csv data/adjacency.csv
```

A numeric adjacency CSV has no row labels or header. The introductory degree closure is undirected; use a properly formulated node model for a directed dataset. The default degree closure uses unweighted topology even when the input carries weights.

6.3 Follow one code path

Start at `cli.run_example`. It loads the graph, computes the degree statistics, resolves the class-wise parameters, calls `solve_degree_control`, and evaluates the resulting policies on the same model. The state and Jacobian functions are together in `heterogeneity.py`; no second sweep implementation is hidden in the entry script.

Listing 4: Run the same mathematical building blocks from Python

```
import networkx as nx
from cybercontrol.graph_data import degree_distribution_from_graph
from cybercontrol.heterogeneity import DegreeSISParams
from cybercontrol.fbs import solve_degree_control

G = nx.barabasi_albert_graph(40, 2, seed=7)
D = degree_distribution_from_graph(G)
model = DegreeSISParams(
    susceptibility=0.12, recovery=0.18,
    state_weight=3.0, control_weight=2.5, control_bound=1.2
).resolve(D.k, D.pk)
result = solve_degree_control(
    D, steps=80, iterations=80, tol=1e-4, model=model
)
print(D.kbar, result.value["cost"])
print(result.value["final_stationarity_residual"])
```

The generic solver obtains its horizon from the named profile. Changing a physical horizon therefore

requires the corresponding profile or solver interface to be updated; changing `steps` only changes the numerical grid.

6.4 Interpret the outputs

The first experiment writes five files to `results/`: `degree_distribution.csv`, `trajectories.csv`, `comparison.csv`, `infection.png`, and `control.png`. The infection plot uses $\sum_k P(k)x_k$. The control plot shows selected degree classes; the trajectory table retains every class with its actual degree label.

The comparison uses no control, the best of nine tested *uniform constant* intensities, and the PMP candidate under the same initial state, model and objective. This constant baseline is not the best arbitrary class-dependent constant vector. Read total cost and infected time together: stronger intervention can lower infection while increasing total cost. A lower cost than these baselines is useful numerical evidence, not a proof of global optimality.

7 Differential games on the same network

7.1 Specify the players and the information structure

An open-loop strategy is a function of time for a specified initial state. A feedback strategy can depend on the currently observed state and time. A Nash profile permits unilateral changes by each player while the others' strategies are fixed. A Stackelberg leader commits first and anticipates a follower's response. These distinctions concern the problem itself, not the order of loops in a numerical solver [5].

For two players maximizing payoffs J_A, J_D, an open-loop Nash profile satisfies

$$J_A(a^*, d^*) \geq J_A(a, d^*), \qquad J_D(a^*, d^*) \geq J_D(a^*, d) \tag{32}$$

for every admissible unilateral deviation. With cost minimization, reverse the inequalities. Opponent controls are held fixed when deriving an open-loop best-response adjoint. Substituting a state-dependent opponent policy changes those derivatives and does not automatically produce a feedback Nash condition.

7.2 A degree-class attacker–defender game

Use

$$\dot{x}_k = \beta(1 + a_k)k(1 - x_k)\Theta - (\delta + d_k)x_k, \tag{33}$$

where $a_k \in [0, \bar{a}_k]$ amplifies acquisition in recipient class k and $d_k \in [0, \bar{d}_k]$ increases recovery. This placement of a_k is part of the model; donor-targeted amplification would have different derivatives. Let

$$\begin{aligned} J_A &= \int_0^T \sum_k P(k) \left[q_A x_k - \frac{c_A}{2} a_k^2 \right] \mathrm{d}t, \\ J_D &= \int_0^T \sum_k P(k) \left[-q_D x_k - \frac{c_D}{2} d_k^2 \right] \mathrm{d}t. \end{aligned} \tag{34}$$

Both players maximize their own payoff; the two payoffs are generally not negatives of each other. Define $H_A = F_A + \lambda_A^\top f$ and $H_D = F_D + \lambda_D^\top f$, where F_A, F_D are the running payoffs. The necessary conditions are one shared state equation and

$$\dot{\lambda}_A = -q_A P - f_x^\top \lambda_A, \qquad \dot{\lambda}_D = q_D P - f_x^\top \lambda_D, \qquad \lambda_A(T) = \lambda_D(T) = 0. \tag{35}$$

Here P denotes the vector of class probabilities. Hamiltonian maximization gives

$$\begin{aligned} a_k^* &= \mathrm{clip}\left(\frac{\lambda_{A,k} \beta k (1 - x_k)\Theta}{c_A P(k)}, 0, \bar{a}_k \right), \\ d_k^* &= \mathrm{clip}\left(-\frac{\lambda_{D,k} x_k}{c_D P(k)}, 0, \bar{d}_k \right). \end{aligned} \tag{36}$$

The sign difference follows from the state effects, not from changing the adjoint convention arbitrarily. Each Hamiltonian is concave in its own scalar effort in this example, but the coupled nonlinear trajectory problem still requires separate equilibrium justification.

The node-level game replaces class pressure by $\sum_j A_{ij}x_j$ and class weights by the chosen node objective weights. Its dimension is the number of nodes, not the number of degree classes. The repository provides both `solve_degree_game` and `solve_node_game`.

7.3 What a numerical game iteration establishes

A coupled forward–backward iteration advances the shared state, solves both adjoints and updates both effort schedules. Convergence supplies a solution of the numerical necessary-condition system, not automatically a Nash equilibrium. A meaningful unilateral-response comparison fixes the opponent and searches over the same admissible strategies, dynamics and payoff.

For a payoff-maximizing player, define

$$\Delta_i = \sup_{v_i} J_i(v_i, u^*_{-i}) - J_i(u^*_i, u^*_{-i}). \tag{37}$$

A profile is an ε-Nash equilibrium when every $\Delta_i \le \varepsilon$. A finite collection of tested deviations, or an imperfectly optimized learned response, gives a *lower bound* on this supremum. Finding an improvement disproves exact equilibrium; failing to find one does not certify a small global gap.

For a Stackelberg game, the leader optimizes through a follower response mapping. Response multiplicity requires a stated selection rule. Updating the leader first in an ordinary Nash sweep does not implement that commitment problem.

8 Dynamic programming, HJB and feedback verification

8.1 The value function and Bellman's argument

For the minimization problem, define the optimal remaining cost

$$V(t,x) = \inf_{u(\cdot)} \left\{ \int_t^T L(x(s), u(s), s)\, \mathrm{d}s + \Phi(x(T)) \right\}. \tag{38}$$

Over a short first interval, dynamic programming gives $V(t,x) = \inf_u \{L\,h + V(t+h, x+hf) + o(h)\}$. If V is differentiable, dividing by h yields

$$V_t + \min_{u\in U}\{L(x,u,t) + \nabla_x V^\top f(x,u,t)\} = 0, \qquad V(T,x) = \Phi(x). \tag{39}$$

A minimizing selector $u^*(t,x)$ is a feedback law defined over a state domain rather than a schedule for one initial state.

Theorem 8.1 (Smooth HJB verification)**.** *Suppose $V \in C^1$ satisfies (39), and a measurable minimizing selector generates admissible trajectories. Then V is the value function and that selector is optimal.*

Proof. For every admissible control, $L + V_t + \nabla V^\top f \ge 0$. Integrating along its trajectory gives $J(u) \ge V(0, x_0)$. Equality holds along the minimizing selector, proving the claim. □

The value function can be nonsmooth. In that case the classical derivatives used here must be replaced by the appropriate generalized/viscosity formulation; fitting a smooth neural function does not establish classical differentiability of the true value.

Along a sufficiently smooth optimal trajectory, $\lambda(t) = \nabla_x V(t, x^*(t))$ relates dynamic programming to PMP. PMP describes one extremal trajectory; HJB describes values and decisions throughout the state domain. This explains both the interpretability of trajectory methods and the dimensional challenge of feedback PDEs.

8.2 Zero-sum and general-sum feedback games

For a zero-sum running payoff ℓ maximized by an attacker and minimized by a defender, the formal Isaacs Hamiltonian is

$$\mathcal{H}(x,p,t) = \max_a \min_d \{\ell(x,a,d,t) + p^\top f(x,a,d,t)\}. \tag{40}$$

The opposite order defines the other game value. Equality of the two pointwise orders is the Isaacs condition. A smooth value satisfying $V_t + \mathcal{H}(x,\nabla V,t) = 0$, together with admissible saddle selectors and the requisite game assumptions, supports feedback verification. The order should never be changed without identifying the information and strategy conventions.

For a general-sum payoff-maximizing game, each player has a value V_i. A smooth feedback Nash candidate satisfies

$$\begin{aligned} 0 &= (V_i)_t + F_i(x,u_i^*,u_{-i}^*,t) + \nabla V_i^\top f(x,u_i^*,u_{-i}^*,t), \\ u_i^* &\in \arg\max_{u_i} \{F_i(x,u_i,u_{-i}^*,t) + \nabla V_i^\top f(x,u_i,u_{-i}^*,t)\}. \end{aligned} \tag{41}$$

These coupled equations retain separate objectives. Minimizing the sum of players' costs generally defines a social-planning problem instead.

9 Impulses, switches and hybrid control

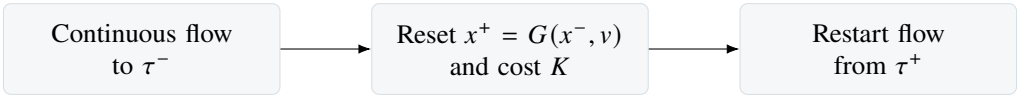
9.1 Describe timing, action values and state effects separately

A sampled real-valued action becomes piecewise constant under zero-order hold. A discrete mode selects a vector field. Neither causes a state jump unless a reset map is specified. In a continuous–impulsive model,

$$\begin{aligned} \dot{x} &= f(x,u,t), \qquad t \neq \tau_\ell, \\ x(\tau_\ell^+) &= G_\ell(x(\tau_\ell^-), v_\ell, \tau_\ell), \end{aligned} \tag{42}$$

and the cost includes $\sum_\ell K_\ell(x(\tau_\ell^-), v_\ell, \tau_\ell)$. For example, an isolation impulse sends $I^+ = (1-v)I^-$ and $R^+ = R^- + vI^-$, while leaving S unchanged. A continuous isolation *rate* is a different model.

To simulate a fixed schedule, integrate exactly to the first event, save the left limit, apply the reset, save the right limit and restart the solver. Repeat at every event. Do not interpolate continuously through a jump or apply its cost on every solver substep.



9.2 Costate jump: derive it from the reset

Let $x^- = x(\tau^-)$ and $x^+ = G(x^-, v, \tau)$. For a fixed event time, a perturbation gives $\delta x^+ = G_x \delta x^- + G_v \delta v$. The first-order event contribution is

$$K_x^\top \delta x^- + K_v^\top \delta v + (\lambda^+)^\top \delta x^+.$$

Matching the coefficient of δx^- to the pre-event costate yields

$$\boxed{\lambda^- = G_x^\top \lambda^+ + K_x.} \tag{43}$$

For a differentiable reset and convex impulse set $\mathcal{V}$, the necessary first-order magnitude condition is

$$\left(K_v + G_v^\top \lambda^+\right)^\top (w - v^*) \geq 0 \quad \text{for all } w \in \mathcal{V}. \tag{44}$$

It becomes equality of the gradient to zero for an interior optimum. For payoff maximization, reverse the inequality, using the corresponding instantaneous payoff.

A finite reset is not a needle variation

The vector λ^+ describes the first-order continuation value. It does not generally justify globally minimizing $K + (\lambda^+)^\top G$ over finite impulse changes. The exact dynamic-programming decision minimizes $K + V^+(\tau, G(x^-, v, \tau))$. A global impulse-Hamiltonian rule needs additional structure; the generic necessary condition is (44).

For a scalar recovery reset $x^+ = (1 - v)x^-$ and $K = cv^2/2$, (43) becomes $\lambda^- = (1 - v)\lambda^+$, and the interior magnitude equation is $cv - \lambda^+ x^- = 0$. The post-event costate depends on the entire resulting trajectory; this is a coupled necessary condition, not a closed-form global optimizer independent of the future.

9.3 A freely chosen event time

If an event time is a free interior decision, the reset and cost are smooth, and no event guard or timing constraint is active, its necessary condition is

$$\boxed{H^- - H^+ + K_t + (\lambda^+)^\top G_t = 0.} \tag{45}$$

The partial derivatives hold the pre-event state and magnitude fixed. To see the sign, differentiate the pre-event accumulated cost plus $K + V^+(\tau, G)$; use (43) and $(V^+)_t = -H^+$. Hamiltonian continuity follows only when both K and G have no explicit time dependence. Guard-triggered switches, minimum spacing, endpoint events and active timing constraints require their corresponding multipliers or one-sided conditions.

For a hybrid game, every player's costate jumps at every event that changes the shared state. Each uses its own instantaneous payoff derivative. The controlling player also satisfies its impulse decision condition. Deriving only the intervening player's adjoint reset would omit the effect on the other players' future values.

9.4 Impulse dynamic programming and the QVI

For optional interventions in a minimization problem, define

$$(\mathcal{M}V)(t, x) = \inf_{v \in \mathcal{V}} \{K(x, v, t) + V(t, G(x, v, t))\}. \tag{46}$$

Immediate intervention is an available choice, so $V \le \mathcal{M}V$. Waiting for a short interval is also available, giving $V_t + \min_u(L + \nabla V^\top f) \ge 0$. At least one choice is locally optimal. Thus the consistent minimization QVI is

$$\boxed{\min\left\{V_t + \min_u(L + \nabla V^\top f),\ \mathcal{M}V - V\right\} = 0.} \tag{47}$$

In the continuation region the differential term is zero and intervention is more expensive. At an intervention point $V = \mathcal{M}V$. Equivalently, use a maximum with *both* terms negated. Mixing a negated differential term with the unnegated obstacle inside a minimum changes the inequalities.

Assume an admissible class excluding infinitely many accumulating interventions and no intervention at T, so $V(T, x) = \Phi(x)$. A different terminal intervention convention needs a matching terminal value. For a fixed scheduled event, the boundary relation is instead $V^-(\tau, x) = \inf_v [K + V^+(\tau, G)]$; optional-intervention QVI logic should not be imposed on a mandatory schedule without adjustment.

9.5 What is optimized in a hybrid computation?

A useful progression fixes event times and optimizes intensities, then considers admissible times for a fixed event count, and finally compares event counts if the application permits it. Each layer needs its own feasibility constraints and cost. Exhaustive or grid search is global only over the finite candidates actually examined. A demonstration that simulates specified impulses is a hybrid simulation, even when its continuous control is sophisticated.

The repository's `simulate_continuous_impulse` illustrates specified flow and reset decisions. The attributed research implementations explore richer impulse searches and games. Their settings should be read before adopting a numerical routine.

10 Extend the model while keeping one mathematical meaning

10.1 Change five connected components together

When replacing a model, update its state equation, state derivatives, objective, adjoint terminal/jump conditions and Hamiltonian optimizer together. The common numerical integrator and graph reader can usually be reused. For a two-state node model, state packing is part of this contract: for example $z = [x_1, \ldots, x_n, y_1, \ldots, y_n]$ must be used consistently by the RHS, Jacobian, costates and output labels.

For awareness-dependent susceptibility, an illustrative extension is

$$\begin{aligned} \dot{x}_i &= \beta_i(1 - y_i)(1 - x_i) \sum_j A_{ij} x_j - (\delta_i + u_i) x_i, \\ \dot{y}_i &= \eta_i(1 - y_i) \sum_j C_{ij} y_j - \mu_i y_i + v_i(1 - y_i). \end{aligned} \tag{48}$$

Here C is the awareness network, y_i is awareness, and v_i increases awareness rather than recovery. Cross-derivatives couple the infection and awareness costates. This is a model-adaptation illustration, not a claim that the default one-state script already solves the two-layer problem.

A delayed equation needs a history function and a delay-aware solver; ordinary `solve_ivp` is an ODE solver, not a general DDE solver [7]. A feedback decision problem with delay may also require a history-based state. Adding a lagged array lookup without specifying those objects does not complete the extension.

10.2 Research code as a next step

The three attributed source snapshots are in `examples/reference/reference_repositories` [19, 17, 18]. Read each network builder before its control routines:

Research example	Code-reading route
Opinion–malware	`network.py` → `opinionMalware.py`: two network layers, forward dynamics, adjoints and impulse choices.
Propaganda war	`demo_network.py` → `propWar.py` → `comparison.py`: degree classes, players, flow and impulse decisions.
Awareness-based propaganda control	`prop_network.py` → `prop_propaganda.py` and comparison scripts: awareness mechanisms, intervention costs and policies.

These are separate research models, with their original licenses and attribution. Their parameters, state meanings and payoffs are not interchangeable. They extend the simple example rather than define extra prerequisites for running it.

10.3 A practical reading order for the tutorial series

This foundation establishes the model and its optimality system. Note 1 turns the same timing and intervention semantics into sampled learning environments and explains learned feedback. Note 2 uses observations and physical equations to infer unknown quantities, and then connects those estimates to control. A coherent application may follow observe–estimate–decide–apply, but uncertainty in the estimated model remains relevant to every subsequent decision.

11 Research examples: connect each method to its modeling purpose

11.1 APT evaluation, patching, repair and recovery

Security evaluation begins by asking how a specified attack–defense mechanism evolves and how its equilibrium reflects security [8]. A control problem adds an admissible intervention and a cost; a differential game further gives a strategic opponent its own decision and objective. The patch-injection and effective-repair papers illustrate this progression [9, 10]. Read the definition of a vulnerable, infected, insecure or repaired state before borrowing a transition: applying recovery control as $-u_i x_i$ and patching susceptible mass as $-u_i S_i$ are different mechanisms.

Backup and recovery introduce an additional operational possibility: information or functionality can be restored from a maintained backup [11]. The choice of state variables must represent the relevant recovery capability, and the objective must distinguish compromise loss from intervention and recovery costs. Simply renaming the recovery rate in an SIS equation does not reproduce a backup model. A useful exercise is to write the state and decision meanings from the paper, identify every controlled transition, and only then form its Hamiltonian.

Impulsive artificial defense studies interventions occurring as events rather than only as continuously applied rates [14]. The outsourcing study introduces an impulsive differential-game setting with distinct decision makers [15]. For either extension, first identify who chooses the continuous effort, event schedule and event intensity. The shared mathematical tools are the forward segments, reset map, costate jump and each player's own objective. The state definitions, event constraints and information structure remain those of the application.

11.2 Conflicting opinions and rumor refutation

The conflicting-opinion study formulates simultaneous benefit maximization for competing opinion processes [12]. The rumor-refuting study links a propagation mechanism to the cost of refutation and strategic interaction [13]. They motivate the use of two objectives rather than automatically setting $J_A = -J_D$. Expenditure by one party is not necessarily a benefit of equal size to the other.

To adapt the degree-game example, replace infection with explicitly defined opinion/rumor states and derive the transitions governing their interaction. If two exclusive states occupy the same population, their feasible region may be $x_k + y_k \leq 1$ rather than two independent intervals. That change affects the RHS and adjoint coupling. The tutorial's one-state attacker–defender example therefore supplies a solution pattern, not the complete opinion or rumor model.

11.3 Propaganda war: degree classes, two players and hybrid strategies

Cheng et al. combine continuous and impulsive interventions in a dynamical propaganda-war game [17]. The accompanying implementation organizes the propagation equations by degree classes. Begin with `demo_network.py`, which reads the networks and computes degree statistics, and continue to `propWar.py`. The forward routines `forwardRed`/`forwardBlue`, backward routines and separate payoff functions reveal how the two players share evolving propagation states while retaining their own objectives.

Trace one iteration as follows: choose both strategies, integrate the coupled state, integrate each player's adjoint under those strategies, update continuous/event decisions, and evaluate the corresponding payoff. Compare this with the simpler continuous degree-game loop in this note. Additional states and impulses require additional derivatives and jump conditions; the population weights and player-specific signs must remain aligned.

The accompanying `comparison.py` fixes one side and samples alternatives for the other. This gives empirical unilateral-deviation comparisons over the sampled strategies. An equilibrium claim over all admissible deviations additionally requires an appropriate theorem or a sufficiently justified best-response

analysis. Its random alternatives should not be interpreted as an exhaustive search.

11.4 Awareness and coupled malware–opinion dynamics

The awareness-based propaganda paper develops impulse interventions for a spreading process influenced by awareness and network topology [18]. Its `prop_network.py` constructs the network matrix; `prop_propaganda.py` supplies propagation, adjoints, policy search and payoff evaluation. The `prop_noAwareness.py` comparison isolates the role of the awareness mechanism. That ablation answers a model question, whereas random intervention comparisons answer a policy question.

The social-engineering malware paper couples malware and opinion/awareness dynamics on a multiplex network and uses separate patching and awareness-training impulse controls [19]. The code stores infection and awareness states separately: `forward_C`, `forward_O`, `backward_dlamb` and `backward_dmu` in `opinionMalware.py` are natural reading anchors. `network.py` builds the layers. A change to one layer can affect the other through cross-state derivatives, even when the networks have different edges.

When adapting the earlier two-state illustration, define both layers' node order, the coupling functions, each event's target and the instantaneous costs. At an event that changes only awareness, the infection state may be continuous while its subsequent derivative changes. This distinction explains why merely plotting a drop in infection at every awareness event would encode the wrong mechanism. Use the publication's actual equations for reproduction; the compact two-layer illustration above is a separately stated teaching model.

11.5 Feedback control: update a model as new information arrives

Jafar et al. connect reinforcement learning with MPC for malware control, using learning to estimate unknown model parameters within prescribed bounds [16]. This provides a bridge between the three tutorials: the present note explains the dynamical control subproblem; Note 1 explains sequential learning and information patterns; Note 2 explains alternative inference and residual-learning constructions.

A generic receding-horizon implementation is

Listing 5: An estimation–control loop; the estimator is application-specific

```
for decision_time in decision_times:
    observation = observe_system()
    estimate = update_state_and_parameters(observation, history)
    plan = solve_finite_horizon_control(estimate, constraints)
    apply_first_interval(plan)
    history.append(observation)
```

This listing describes the interface, not the detailed algorithm of the cited paper. Replacing its estimator by a PINN or its controller by a learned policy is a new methodological choice. Evaluate estimation and decision quality separately so that a benefit from better information is not automatically attributed to a new optimizer or policy learner.

A Short exercises with worked answers

Direction. A directed dataset contains only `A,B,2`. With node order (A, B), what is the incoming-influence matrix?

Answer: $A = \left[\begin{smallmatrix} 0 & 0 \\ 2 & 0 \end{smallmatrix}\right]$. Node B receives pressure $2x_A$.

Control weights. Why does $P(k)$ appear in the denominator of (23)?

Answer: The intervention cost is $P(k)r_k u_k^2/2$, whereas the costate term uses the unweighted state equation.

Differentiating gives $P(k)r_k u_k - \lambda_k x_k$. Rescaling costates by class weights changes the adjoint equations as well as the update.

Finite impulse changes. Let $G(v) = v$, $v \in [-1, 1]$, $K(v) = 1 - v^2 + v^4$, and terminal cost $\Phi(x) = x^2$. Does minimizing $K + \lambda^+ G$ recover the actual optimum?

Answer: The total cost is $1+v^4$, minimized at $v = 0$, with $\lambda^+ = \Phi'(0) = 0$. The surrogate $K+\lambda^+ G = 1-v^2+v^4$ is minimized at $v = \pm 1/\sqrt{2}$. The first-order condition at zero remains valid; the claimed global surrogate minimum does not.

Boundary optimality. For $H(u) = u^2/2 + 2u$ and $0 \le u \le 1$, evaluate the optimum and projected residual.

Answer: The optimum is $u = 0$ although $H_u(0) = 2$. With step size one, $u - \mathrm{clip}(u - H_u, 0, 1) = 0$ at that optimum. The raw derivative penalty would incorrectly prefer moving away from the valid bound.

B Where to run extensions and numerical checks

The ordinary entry point remains `python -m cybercontrol`. For the existing broader degree, node, game and hybrid examples, run

```
python -m cybercontrol study --output-dir results/extensions
```

For development, install `python -m pip install -e ".[dev]"` and run `pytest -q`. The public `check` command runs a small numerical check. Only `python -m cybercontrol docs` requires LaTeX/`latexmk`. Numerical correctness checks support the computation; scientific conclusions still depend on the model, data and comparison problem.

References

[1] L. S. Pontryagin, V. G. Boltyanskii, R. V. Gamkrelidze and E. F. Mishchenko, *The Mathematical Theory of Optimal Processes*. Wiley, 1962.

[2] D. Liberzon, *Calculus of Variations and Optimal Control Theory: A Concise Introduction*. Princeton University Press, 2012. Author's course text: https://liberzon.csl.illinois.edu/teaching/cvoc/.

[3] S. Lenhart and J. T. Workman, *Optimal Control Applied to Biological Models*. Chapman and Hall/CRC, 2007.

[4] M. McAsey, L. Mou and W. Han, "Convergence of the forward-backward sweep method in optimal control," *Computational Optimization and Applications*, 53, 207–226, 2012.

[5] T. Başar and G. J. Olsder, *Dynamic Noncooperative Game Theory*, second edition. SIAM, 1999.

[6] NetworkX developers, *Converting to and from other data formats*, official documentation. https://networkx.org/documentation/stable/reference/convert.html.

[7] SciPy developers, *solve_ivp*, official documentation. https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.solve_ivp.html.

[8] L.-X. Yang, P. Li, X. Yang and Y. Y. Tang, "Security evaluation of the cyber networks under advanced persistent threats," *IEEE Access*, 5, 20111–20123, 2017. doi:10.1109/ACCESS.2017.2757944.

[9] L.-X. Yang, P. Li, X. Yang, Y. Xiang and W. Zhou, "A differential game approach to patch injection," *IEEE Access*, 6, 58924–58938, 2018. doi:10.1109/ACCESS.2018.2874446.

[10] L.-X. Yang, P. Li, Y. Zhang, X. Yang, Y. Xiang and W. Zhou, "Effective repair strategy against advanced persistent threat: A differential game approach," *IEEE Transactions on Information Forensics and Security*, 14(7), 1713–1728, 2019. doi:10.1109/TIFS.2018.2885251.

[11] L.-X. Yang, K. Huang, X. Yang, Y. Zhang, Y. Xiang and Y. Y. Tang, "Defense against advanced persistent threat through data backup and recovery," *IEEE Transactions on Network Science and Engineering*, 8(3), 2001–2013, 2021. doi:10.1109/TNSE.2020.3040247.

[12] L.-X. Yang, P. Li, X. Yang, Y. Xiang and Y. Y. Tang, "Simultaneous benefit maximization of conflicting opinions: Modeling and analysis," *IEEE Systems Journal*, 14(2), 1623–1634, 2020. doi:10.1109/JSYST.2020.2964004.

[13] D.-W. Huang, L.-X. Yang, P. Li, X. Yang and Y. Y. Tang, "Developing cost-effective rumor-refuting strategy through game-theoretic approach," *IEEE Systems Journal*, 15(4), 5034–5045, 2021. doi:10.1109/JSYST.2020.3020078.

[14] H. Sun, X. Yang, L.-X. Yang, K. Huang and G. Li, "Impulsive artificial defense against advanced persistent threat," *IEEE Transactions on Information Forensics and Security*, 18, 3506–3516, 2023. doi:10.1109/TIFS.2023.3284564.

[15] Y. Qin, X. Yang, L.-X. Yang and K. Huang, "Modeling and study of defense outsourcing against advanced persistent threat through impulsive differential game approach," *Computers & Security*, 145, 104003, 2024. doi:10.1016/j.cose.2024.104003.

[16] M. T. Jafar, L.-X. Yang, G. Li, Q. Zhu and C. Gan, "Minimizing malware propagation in Internet of Things networks: An optimal control using feedback loop approach," *IEEE Transactions on Information Forensics and Security*, 19, 9682–9697, 2024. doi:10.1109/TIFS.2024.3463965.

[17] X. Cheng, L.-X. Yang, Q. Zhu, C. Gan, X. Yang and G. Li, "Cost-effective hybrid control strategies for dynamical propaganda war game," *IEEE Transactions on Information Forensics and Security*, 19, 9789–9802, 2024. doi:10.1109/TIFS.2024.3468903. Author code: PropagandaWar_TIFS_2024_Code.

[18] X. Cheng, L.-X. Yang, Q. Zhu, C. Gan and G. Li, "Impulse strategies for suppressing cyber propaganda with awareness," *IEEE Transactions on Computational Social Systems*, 12(5), 2685–2695, 2025. doi:10.1109/TCSS.2024.3522889. Author code: Propaganda_TCSS_2025_Code.

[19] X. Cheng, L.-X. Yang, G. Li, Z. Ma, T. Zhu, L. Wang and S. Duan, "Modeling and mitigating social engineering malware: Integrating malware-opinion dynamics with optimal impulse control approaches," *IEEE Transactions on Information Forensics and Security*, 20, 12231–12244, 2025. doi:10.1109/TIFS.2025.3630005. Author code: OpinionMalware_TIFS_2025_Code.